

CUNEF

UNIVERSIDAD

Browser · Web Application Programming

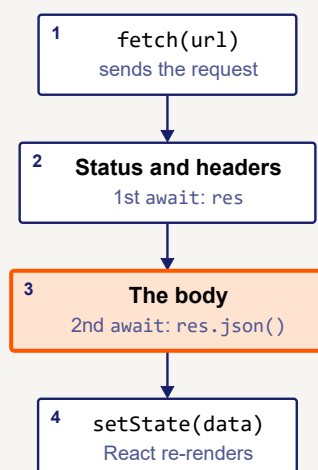
Fetching data: summary

1. **Requests and responses:** fetch: a request in two waits · fetch does not fail on 404 or 500: check `res.ok` · POST, PUT, PATCH and DELETE from a form
2. **Loading data for the screen:** Where the call goes: an effect or an event handler · Race conditions: the stale response · Cancelling with `AbortController` · Loading, error, empty, success
3. **A data hook, search and pages:** A reusable data hook: `{ data, loading, error, reload }` · Debounced search · Pagination controls: page, limit, total
4. **After a write:** After a write: refetch, or update local state · Optimistic updates (and rolling back) · Server validation errors on the right field
5. **One API client:** Environment variables in the front end (and why they are public) · One API client module · fetch vs Axios: what a library adds · The dev proxy: `/api` → `localhost:3000` without CORS
6. **Logging in and tokens:** Logging in from the front end · Sending the token: Authorization: Bearer · Handling 401: log out and go to the log-in page

Requests and responses

fetch: a request in two waits

`fetch(url, options)` sends an HTTP request from the browser and returns a **promise of a Response**; reading the body with `await res.json()` is a **second promise**.



Two waits: first the status and headers, then the body.

`const res = await fetch(url)` Status and headers: `res.status`, `res.ok`, `res.headers.get(name)`.

`await res.json()` The body, parsed with `JSON.parse` (see [JSON](#)); `res.text()` gives it as a string. A body can be read once.

The options

With none, fetch sends a GET. The second argument sets method, headers, body and signal (see [Writes from a form](#) and [Cancelling with AbortController](#)).

- **The page keeps running:** `await` pauses your function, not the page. When the answer arrives you call a state setter, and React re-renders with the data.
- **Relative URLs** such as `/api/tasks` go to the page's own origin; in development the dev server forwards `/api` to the API (see [The dev proxy](#)).
- 204 No Content has no body: calling `res.json()` on it throws.
- **Watch every request** in the Network tab of DevTools (see [The Network tab](#)).

```

import { useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [info, setInfo] = useState('');

```

```

async function load() {
  const res = await fetch('/api/tasks');           // wait 1: status and headers
  console.log('status', res.status, 'ok', res.ok);
  console.log('X-Total-Count', res.headers.get('X-Total-Count'));
  const data = await res.json();                   // wait 2: the body, parsed as JSON
  setInfo(res.status + ' ' + res.statusText + ': ' + data.length + ' tasks');
  setTasks(data);                                 // new state: React re-renders
}

return (
  <main>
    <button onClick={load}>Load tasks</button>
    <p>{info}</p>
    <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
  </main>
);
}

```

Example. In the Try it box, press Load tasks: the console shows the status and the X-Total-Count header after the first await, and the list appears once the body is parsed. Every Try it box in this section talks to a small mock API of tasks that answers /api/... (it starts afresh when you edit the code).

Common mistake. Writing `const tasks = fetch('/api/tasks').json()`. `fetch` returns a promise, which has no `.json()` method (TypeError), and even `(await fetch(url)).json()` is still a promise: you need both awaits, `const res = await fetch(url); const tasks = await res.json();`.

fetch does not fail on 404 or 500: check res.ok

The promise of `fetch` **rejects only when no response arrives** (network down, CORS block, cancelled); a 404 or a 500 is a response, so it **resolves**, and you check `res.ok` (status 200–299) yourself.

What fetch does in each case

What happened	Promise	res.ok	Your catch runs?
200 with the task	Resolves	true	No (nothing to catch)
404, 401 or 500	Resolves	false	Only if you throw
Network down, server not running	Rejects (TypeError: Failed to fetch)	No response	Yes
Blocked by CORS	Rejects (TypeError)	No response for your code	Yes
Cancelled with AbortController	Rejects (AbortError)	No response	Yes

```

import { useState } from 'react';

async function getTask(id) {
  const res = await fetch('/api/tasks/' + id);
  if (!res.ok) {
    // 404 and 500 RESOLVE: turn them into errors ourselves, with the server's message
    const body = await res.json().catch(() => ({}));
    throw new Error(body.error || 'HTTP ' + res.status);
  }
  return res.json();
}

function App() {
  const [id, setId] = useState('2');
  const [result, setResult] = useState('');

  async function show() {
    try {
      const task = await getTask(id);
      setResult('Found: ' + task.title);
    } catch (err) {
      setResult('Error: ' + err.message);
    }
  }
}

```

```

return (
  <main>
    <label>Task id <input value={id} onChange={(e) => setId(e.target.value)} /></label>
    <button onClick={show}>Fetch</button>
    <p>{result}</p>
  </main>
);
}

```

Example. In the Try it box, fetch task 2 (found) and task 99 (the helper turns the 404 into an error with the server's message). Then delete the whole `if (!res.ok) { ... }` block and fetch 99 again: the page says "Found: undefined", because the error body `{ error: ... }` was treated as a task.

Common mistake. Wrapping a fetch in try/catch and believing errors are handled. Without a `res.ok` check, a 401 or a 500 sails past the catch, the error body is stored as if it were data, and the UI breaks somewhere else ("tasks.map is not a function", because `{ error }` is not an array).

POST, PUT, PATCH and DELETE from a form

A write is a fetch with a method, a Content-Type: application/json header and a body made with JSON.stringify; the form's onSubmit handler calls e.preventDefault() and sends it.

method POST to create in a collection, PUT to replace, PATCH to change some fields, DELETE to remove (see [CRUD on HTTP](#)).

Content-Type: application/json Tells the server how to read the body. Without it, the server's JSON parser skips the body and validation answers "title is required" (see [express.json\(\)](#) and [req.body](#)).

body: JSON.stringify(data) A body is always text (see [JSON](#)).

Reading the answer by its status

Status	Body	Do
201 Created	The new resource, with the id and fields the server set	Use it
200 OK	The updated resource	Use it
204 No Content	None	Do not call <code>res.json()</code>
400 Bad Request	The validation errors	Show them on the fields (see Server validation errors)

```

import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState('');

  useEffect(() => {
    fetch('/api/tasks').then((res) => res.json()).then(setTasks);
  }, []);

  async function add(e) {
    e.preventDefault(); // no page reload
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' }, // "the body is JSON"
      body: JSON.stringify({ title }), // object → JSON text
    });
    if (!res.ok) return console.log('not created:', res.status);
    const created = await res.json(); // 201: the task, with its id
    setTasks([...tasks, created]);
    setTitle('');
  }

  async function remove(id) {
    const res = await fetch('/api/tasks/' + id, { method: 'DELETE' });
    if (res.status === 204) setTasks(tasks.filter((t) => t.id !== id)); // 204: no body
  }
}

```

```

}

return (
  <form onSubmit={add}>
    <input value={title} onChange={(e) => setTitle(e.target.value)} aria-label="New task" />
    <button>Add</button>
    <ul>{tasks.map((t) => (
      <li key={t.id}>{t.title} <button type="button" onClick={() => remove(t.id)}>Delete</button></li>
    ))}</ul>
  </form>
);
}

```

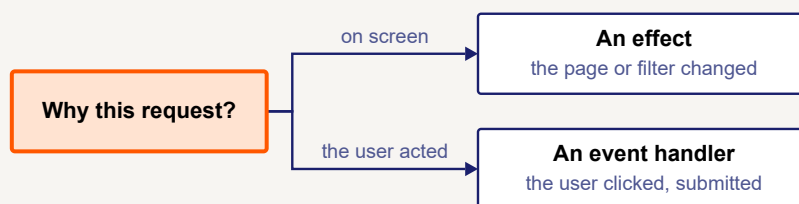
Example. In the Try it box, type a title and press Enter or Add: a POST (201) adds it with its new id. Delete sends DELETE /api/tasks/:id and gets 204, so the code removes the item from the list without parsing any body. The Delete buttons have type="button": inside a form, a plain <button> is a submit button.

Common mistake. Sending body: { title } (an object) instead of body: JSON.stringify({ title }). fetch converts the object to the string "[object Object]", and the server answers 400 for malformed JSON. A body is always text.

Loading data for the screen

Where the call goes: an effect or an event handler

Fetch in an **effect** when the data is needed because something is **on screen** (the page opened, a filter changed); fetch in an **event handler** when the request is the result of something the user **did** (submit, delete, mark as done).



Reads follow the screen; writes follow the user.

- **Never in the component body:** the body runs on every render, the response sets state, the new state renders again: an endless loop of requests.
- **The effect itself** (why it cannot be async, its dependencies, its cleanup) is in [Fetching data in an effect](#) and [The dependency array](#). The ignore flag in the Try it box is explained in [Race conditions](#).
- **No writes in effects:** an effect that POSTs "when submitted becomes true" is harder to follow than a handler, and in development [StrictMode](#) runs effects twice: harmless for a GET with a cleanup, a duplicate row for a POST.
- **Data libraries** (TanStack Query, SWR) and router loaders move reads out of effects and add caching (see [Data-fetching caches](#)); they rest on the same ideas.

```

import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [showDone, setShowDone] = useState(false);

  // Needed because it is on screen: an effect that runs again when showDone changes.
  useEffect(() => {
    let ignore = false;
    fetch('/api/tasks?done=' + showDone)
      .then((res) => res.json())
      .then((data) => { if (!ignore) setTasks(data); });
    return () => { ignore = true; };
  });
}

```

```

    }, [showDone]);

    // Caused by a click: the request goes in the event handler.
    async function toggle(task) {
      const res = await fetch('/api/tasks/' + task.id, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ done: !task.done }),
      });
      if (res.ok) setTasks(tasks.filter((t) => t.id !== task.id)); // it moved to the other list
    }

    return (
      <main>
        <label><input type="checkbox" checked={showDone} onChange={(e) => setShowDone(e.target.checked)} /> Show done
        tasks</label>
        <ul>{tasks.map((t) => (
          <li key={t.id}>{t.title} <button onClick={() => toggle(t)}>{t.done ? 'Undo' : 'Done'}</button></li>
        ))}</ul>
      </main>
    );
  }
}

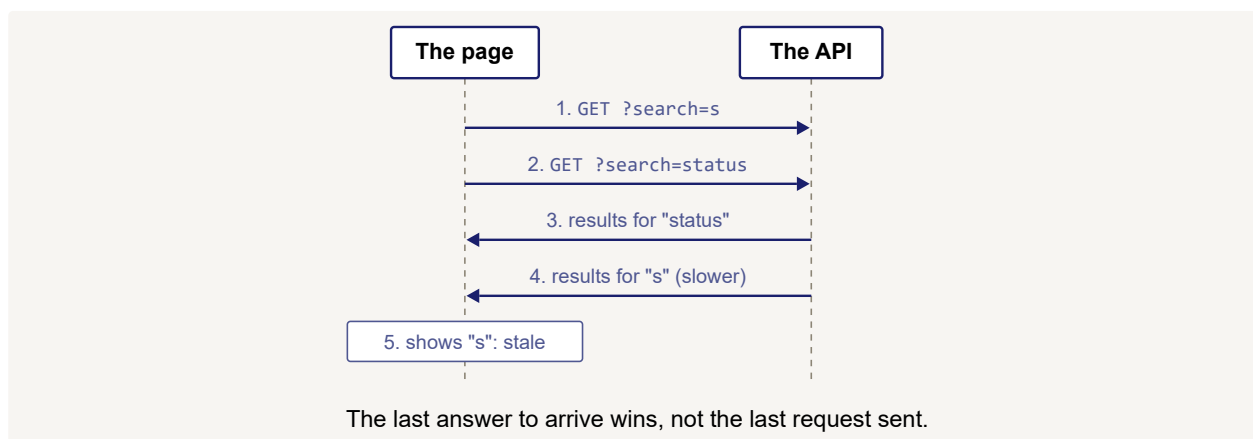
```

Example. In the Try it box, the list is fetched in an effect that depends on `showDone`: tick the checkbox and the effect runs again with `?done=true`. Marking a task as done is a user action, so the PATCH is sent from the button's `onClick`, which also knows which task to change.

Common mistake. Leaving a dependency out of the array (`useEffect(load, [])` when `load` reads `showDone`). The effect runs once with the first value and never again, so the filter "does nothing". List every value from the component that the effect reads (see [The dependency array](#)).

Race conditions: the stale response

A **race condition** happens when two requests are in flight and the answers arrive in a different order than the requests left: the **older (stale)** response lands last and overwrites the newer one, so the screen shows results for something the user no longer asked for.



The fix: only the current request may set state

- **The ignore flag:** `let ignore = false` in the effect, `ignore = true` in its cleanup, and `if (ignore) return;` before the setter. React runs the cleanup before the effect runs again and on unmount (see [Cleanup](#)), so a stale answer finds `ignore === true` and does nothing.
- **Or cancel the old request** (see [Cancelling with AbortController](#)).
- **The same bug, elsewhere:** a detail page that fetches by the `:id` in the URL, tabs, pagination. The cleanup costs two lines: write it in every effect that fetches by a changing value.

```
import { useEffect, useState } from 'react';
```

```
// Broad searches are slower on a real server; here we simulate it: "s" waits 1.5 s more than "status".
const slowFor = (q) => Math.max(0, 6 - q.length) * 300;
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function App() {
  const [query, setQuery] = useState('');
  const [tasks, setTasks] = useState([]);
  const [shownFor, setShownFor] = useState('');

  useEffect(() => {
    let ignore = false; // the fix, part 1
    async function search() {
      const res = await fetch('/api/tasks?search=' + encodeURIComponent(query));
      const data = await res.json();
      await wait(slowFor(query));
      if (ignore) return; // the fix, part 2: delete this line to see the bug
      setTasks(data);
      setShownFor(query);
    }
    search();
    return () => { ignore = true; }; // the query changed: this answer is now stale
  }, [query]);

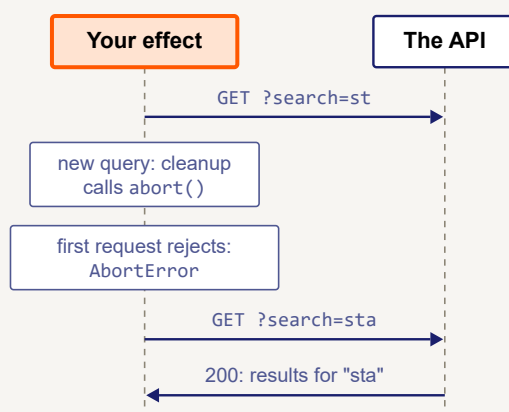
  return (
    <main>
      <label>Type "status" quickly: <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      <p>Box: "{query}" · list shows results for: "{shownFor}"</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. The Try it box simulates a server where short searches are slower (300 ms more per missing letter). Type "status" quickly: the list ends up right. Now delete the line `if (ignore) return;` and type again: the list jumps around and can end with the results for "s" while the search box says "status". The Fetch lab draws each request on a timeline.

Common mistake. Believing that requests sent in order come back in order, because it works on your machine where the API answers in 5 ms. On a real network with real data the order is not guaranteed, and the bug shows up only for users. Protect every effect that fetches by a changing value.

Cancelling with AbortController

An **AbortController** **cancels** a fetch: **pass** `controller.signal` **in the options**, **call** `controller.abort()` **in the effect's cleanup**, **and the request's promise rejects with an error named** `AbortError`, **which you ignore**.



Each new query cancels the request before it.

1. Create a controller inside the effect: `const controller = new AbortController();`
2. Pass its signal: `fetch(url, { signal: controller.signal })`.

3. `Return () => controller.abort()` as the cleanup. React calls it when the dependency changes (cancel the old request) and when the component unmounts.
4. In the catch, return early when `err.name === 'AbortError'`: you changed your mind, which is not an error for the user.

The ignore flag or `AbortController`?

	The ignore flag	<code>AbortController</code>
The old request	Completes; its result is thrown away	Is cancelled by the browser
Saves	Nothing	Bandwidth and waiting (the server may still have done the work)
Works with	Any promise	<code>fetch</code> , <code>Axios</code> and most libraries; one signal can cancel several requests

```
import { useEffect, useState } from 'react';

function App() {
  const [query, setQuery] = useState('');
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setError(null);
    fetch('/api/tasks?search=' + encodeURIComponent(query), { signal: controller.signal })
      .then((res) => {
        if (!res.ok) throw new Error('HTTP ' + res.status);
        return res.json();
      })
      .then(setTasks)
      .catch((err) => {
        if (err.name === 'AbortError') return; // we cancelled it on purpose: not an error
        setError(err);
      });
    return () => controller.abort(); // the query changed, or the component left
  }, [query]);

  return (
    <main>
      <label>Search <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      {error && <p role="alert">{error.message}</p>}
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}
```

Example. In the Try it box (each answer takes 0.8 s), type a word quickly and read the console: every keystroke cancels the request before it ("→ cancelled"), and only the last one gets a 200.

Common mistake. Creating one `AbortController` outside the effect (or in a `useRef`) and reusing it. Once aborted, a controller stays aborted: every later request using its signal is cancelled immediately. Create a new controller in each effect run.

Loading, error, empty, success

A component that loads data is always in one of **four states**: **loading** (no answer yet), **error** (it failed), **empty** (it worked, and there is nothing), **success** (there is data); render each one on purpose.

Four states, four renders

State	When	Show
Loading	Request sent, no answer yet	"Loading tasks..." or a skeleton, in a region with <code>aria-busy="true"</code> ; disable buttons that would repeat it
Error	A rejected promise, or <code>res.ok === false</code>	What failed, in words, with <code>role="alert"</code> and a Try again button
Empty	Success with <code>[]</code>	Why it is empty and what to do next ("No tasks yet: create the first one")

State	When	Show
Success	Success with items	The data

Model it so nonsense cannot happen

- **One status field** ('loading' | 'error' | 'success') plus the data and the error. Separate booleans allow "loading and error" at once, and every branch must check them all.
- **Empty is derived:** success with zero items, checked inside the success branch, not a fourth result.
- **Render in order:** loading, then error, then empty, then the list. Keep the layout stable: a search box stays on screen while the results load.
- **Start in 'loading':** the first render happens before the request has even started.

```
import { useEffect, useState } from 'react';

const SEARCH = ''; // try 'zzz' for the empty state

function App() {
  const [state, setState] = useState({ status: 'loading', tasks: [], error: null });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let ignore = false;
    setState({ status: 'loading', tasks: [], error: null });
    fetch('/api/tasks?search=' + SEARCH)
      .then((res) => {
        if (!res.ok) throw new Error('HTTP ' + res.status);
        return res.json();
      })
      .then((tasks) => { if (!ignore) setState({ status: 'success', tasks, error: null }); })
      .catch((error) => { if (!ignore) setState({ status: 'error', tasks: [], error }); });
    return () => { ignore = true; };
  }, [attempt]);

  if (state.status === 'loading') return <p aria-busy="true">Loading tasks...</p>;
  if (state.status === 'error') return (
    <p role="alert">Could not load the tasks ({state.error.message}).{' '}
      <button onClick={() => setAttempt(attempt + 1)}>Try again</button></p>
  );
  if (state.tasks.length === 0) return <p>No tasks match. Create your first one!</p>;
  return <ul>{state.tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
}
```

Example. The Try it box waits 1.2 s per request, so you see the loading state. Change SEARCH to 'zzz' for the empty state. For the error state, open the Fetch lab, which has switches for a network failure and a 500.

Common mistake. Showing "No tasks" while the request is still running. If the initial state is tasks = [] and you only check tasks.length === 0, the empty message flashes before every load and also appears after an error. Decide the state first (loading? error?), and only then look at the length.

A data hook, search and pages

A reusable data hook: { data, loading, error, reload }

A custom hook that wraps the loading pattern (status, data, error, abort, reload) and returns { data, loading, error, reload } lets every component load data in one line: `useApi('/api/tasks')`.

The URL is the dependency

When it changes (a new id, a new search), the hook loads again and aborts the previous request. Keep dependencies primitive: a URL string, an id.

reload

Bumps a version counter listed in the dependencies, so the effect runs again on demand: a Try again button, or after creating an item.

Each call has its own state

Two components calling `useApi('/api/tasks')` send two requests and hold two copies; sharing one copy is the job of a cache.

Headers and base URL

Belong in the API client the hook calls, not in the hook.

```
import { useEffect, useState } from 'react';

// The loading / error / data logic, written once.
function useApi(path) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [version, setVersion] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);
    fetch(path, { signal: controller.signal })
      .then((res) => { if (!res.ok) throw new Error('HTTP ' + res.status); return res.json(); })
      .then((json) => { setData(json); setLoading(false); })
      .catch((err) => { if (err.name !== 'AbortError') { setError(err); setLoading(false); } });
    return () => controller.abort();
  }, [path, version]);

  return { data, loading, error, reload: () => setVersion((v) => v + 1) };
}

function TaskCount() {
  const { data, loading } = useApi('/api/tasks?done=false');
  return <p>{loading || !data ? '...' : data.length} tasks to do</p>;
}

function App() {
  const { data, loading, error, reload } = useApi('/api/tasks');
  return (
    <main>
      <TaskCount />
      <button onClick={reload} disabled={loading}>Reload</button>
      {error && <p role="alert">{error.message}</p>}
      {data && <ul>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul>}
    </main>
  );
}
```

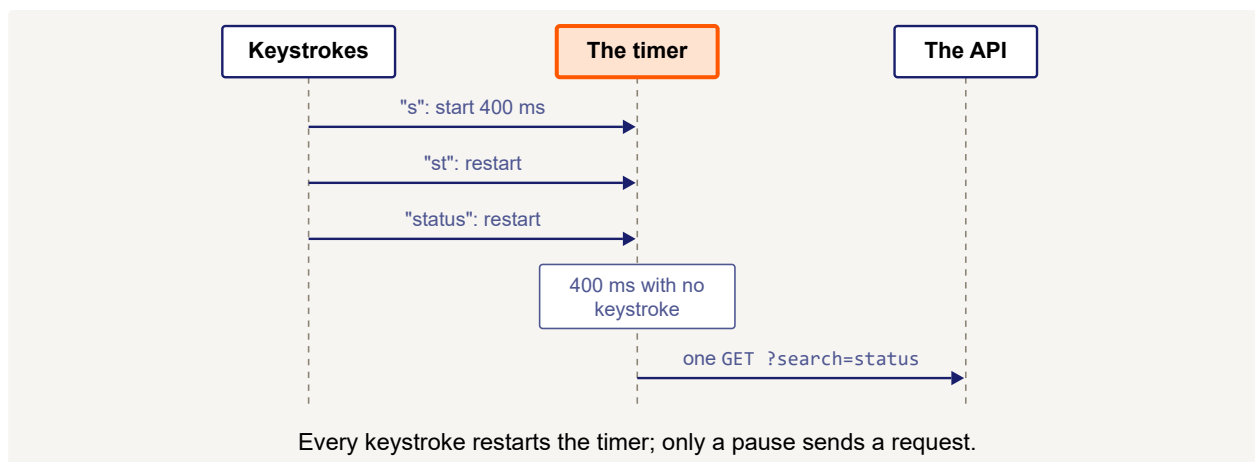
Example. The Try it box defines `useApi` once and uses it twice: `TaskCount` shows how many tasks are not done, `App` shows the whole list with a Reload button. Each call has its own loading state and its own request (see the console).

Common mistake. Passing an options object or an array that is recreated on every render, and listing it as a dependency: the effect then runs after every render, an endless request loop. Keep dependencies primitive. A related slip: calling the hook conditionally (if (id) useApi(...)). Hooks run in the same order on every render, so pass `null` and let the effect return early.

Debounced search

Debouncing waits until the input has stopped changing for a short time (say 300–500 ms) before acting, so a

search box sends **one** request when the user pauses instead of one per keystroke.



- **A small hook**, `useDebounce(value, ms)`, returns the value once it has been stable for `ms`: its effect sets a timer and its cleanup clears it (see [Cleanup](#)).
- **Fetch by the debounced value**, and keep the input controlled by the raw value, so typing stays instant.
- **Keep the abort or ignore cleanup**: a pause, more typing and a slow answer can still overlap.
- **Or search on submit** (a button or Enter): fewer surprises, one more click. Choose on purpose.

```

import { useEffect, useState } from 'react';

// Gives back `value`, but only once it has stopped changing for `ms`.
function useDebounce(value, ms) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), ms);
    return () => clearTimeout(timer); // a new keystroke restarts the wait
  }, [value, ms]);
  return debounced;
}

function App() {
  const [query, setQuery] = useState('');
  const search = useDebounce(query, 400);
  const [tasks, setTasks] = useState([]);
  const [requests, setRequests] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setRequests((n) => n + 1);
    fetch('/api/tasks?search=' + encodeURIComponent(search), { signal: controller.signal })
      .then((res) => res.json())
      .then(setTasks)
      .catch((err) => { if (err.name !== 'AbortError') console.error(err); });
    return () => controller.abort();
  }, [search]);

  return (
    <main>
      <label>Search <input type="search" value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      <p>Requests sent: {requests}{query !== search && ' · waiting for you to stop typing...'}</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}

```

Example. In the Try it box, type "status" quickly: the counter goes up by just one, after you stop (the first request is the empty search when the Try it box loads). Type slowly, one letter per second, and every letter becomes a request. Change 400 to 0 and every keystroke sends one again.

Common mistake. Writing `const search = debounce(fetchTasks, 400)` in the body of the component. The body runs on every render, so each keystroke creates a **new** debounced function with its own timer, and nothing is debounced at all. Keep the timer in an effect (as the Try it box does) or in a [useRef](#).

Pagination controls: page, limit, total

A paginated list keeps the current **page** in state, asks the API for `?page=&limit=`, reads the **total** the API reports, and computes the number of pages (`Math.ceil(total / limit)`) to show **"Page 2 of 3"** and to **disable Previous** on the first page and **Next** on the last.

Pages `Math.max(1, Math.ceil(total / limit))`

"Showing 3—4 of 5" `From (page - 1) × limit + 1 to Math.min(page × limit, total).`

Previous and Next Disabled when `page <= 1` and `page >= pages`, and both while a page loads, so a double click does not skip one.

- **Back to page 1** when the search or a filter changes: page 3 of the old results may not exist.
- **page is a dependency** of the effect (`[page, search]`), with the usual cleanup against races.
- **The server caps** `limit` to protect itself (see [Pagination limits as protection](#)).
- **In the URL** (`?page=2`), a reload or a shared link opens the same page (see [Query strings: useSearchParams](#)).

```
import { useEffect, useState } from 'react';

const LIMIT = 2;

function App() {
  const [page, setPage] = useState(1);
  const [result, setResult] = useState({ items: [], total: 0 });
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let ignore = false;
    setLoading(true);
    fetch('/api/tasks?page=' + page + '&limit=' + LIMIT)
      .then(async (res) => {
        const total = Number(res.headers.get('X-Total-Count')); // this API sends the total in a header
        const items = await res.json();
        if (!ignore) { setResult({ items, total }); setLoading(false); }
      });
    return () => { ignore = true; };
  }, [page]);

  const pages = Math.max(1, Math.ceil(result.total / LIMIT));
  const from = result.total === 0 ? 0 : (page - 1) * LIMIT + 1;
  const to = Math.min(page * LIMIT, result.total);

  return (
    <main aria-busy={loading}>
      <ul>{result.items.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
      <p>Showing {from}—{to} of {result.total}</p>
      <nav aria-label="Pagination">
        <button onClick={() => setPage(page - 1)} disabled={page <= 1 || loading}>< Previous</button>
        { ' ' }Page {page} of {pages}{ ' ' }
        <button onClick={() => setPage(page + 1)} disabled={page >= pages || loading}>Next ></button>
      </nav>
    </main>
  );
}
```

Example. The Try it box asks for two tasks per page. Five tasks make three pages: on page 1 Previous is disabled; on page 3 ("Showing 5–5 of 5") Next is disabled. While a page loads (0.6 s here) both are disabled.

Common mistake. Deciding that "Next" is possible when the page came back full (`items.length === limit`). On the last page that happens to be full (4 items, 2 per page), Next stays enabled and leads to an empty page. Use the total: `page < Math.ceil(total / limit)`.

After a write

After a write: refetch, or update local state

When a write succeeds, the screen must show the change: either **refetch** the data from the server (simple, always correct, one more request) or **update local state** from the response (fast, but you must reproduce the server's rules: sorting, filters, counts).

Two ways to show a write

	Refetch	Update local state
Requests	One more (a GET)	None: use the 201 or 200 body
Correct	Always, including other users' changes	As long as you reproduce the server's sorting, filters and counts
Best when	The server decides where things go (sorted, filtered, paginated)	The change is obvious: append, remove, replace one item by id

- **Use the updater form** (`setTasks((prev) => [created, ...prev])`): the write took time, and `prev` is the state now, not the state when the click happened (see [Updater functions](#)).
- **Data libraries** call refetching "invalidating" a query and make it one line (see [Data-fetching caches](#)).

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [strategy, setStrategy] = useState('local');

  const load = () => fetch('/api/tasks?sort=createdAt&order=desc').then((r) => r.json()).then(setTasks);
  useEffect(() => { load(); }, []);

  async function add() {
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'Added at ' + new Date().toLocaleTimeString() }),
    });
    if (!res.ok) return;
    if (strategy === 'refetch') {
      await load(); // a second request: the server's truth
    } else {
      const created = await res.json(); // use the 201 body: no second request
      setTasks((prev) => [created, ...prev]); // newest first, as the server sorts
    }
  }

  return (
    <main>
      <label><input type="radio" checked={strategy === 'local'} onChange={() => setStrategy('local')} /> Update local state</label>
      <label><input type="radio" checked={strategy === 'refetch'} onChange={() => setStrategy('refetch')} /> Refetch the list</label>
      <p><button onClick={add}>Add a task</button> (watch the [network] lines below)</p>
      <ol>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ol>
    </main>
```

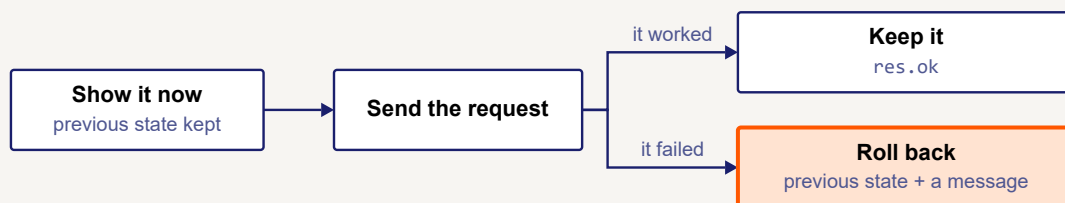
```
);
}
```

Example. In the Try it box, switch between the two strategies and add a few tasks. "Update local state" sends one request (POST) and puts the new task first, as the server sorts; "Refetch the list" sends a POST and then a GET. Both show the same list; the console shows the cost.

Common mistake. Building the new item on the client (`setTasks([...tasks, { title }])`) instead of using the server's response. It has no `id` (so no key and nothing to delete later), no `createdAt`, and it shows even when the server rejected the write. Wait for success and use what the server returned.

Optimistic updates (and rolling back)

An **optimistic update** changes the screen **before** the server answers, assuming success, and **rolls back** to the previous state, with a message, if the request fails.



Bet on success, and keep a way back.

1. Remember the previous state.
2. Show the change.
3. Send the request and check `res.ok`.
4. On failure, restore the previous state and say what did not happen. A silent rollback is confusing (the tick jumps back for no visible reason); a silent failure is worse (the screen lies until the next reload).

```
import { useEffect, useState } from 'react';

function App() {
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState('');

  useEffect(() => {
    fetch('/api/tasks').then((r) => r.json())
      // A task someone else deleted a minute ago: our list still shows it.
      .then((data) => setTasks([...data, { id: 99, title: 'Deleted elsewhere', done: false }]]));
  }, []);

  async function toggle(task) {
    const before = tasks; // 1. remember
    setTasks(tasks.map((t) => (t.id === task.id ? { ...t, done: !t.done } : t))); // 2. show it now
    setError('');
    try {
      const res = await fetch('/api/tasks/' + task.id, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ done: !task.done }),
      });
      if (!res.ok) throw new Error('HTTP ' + res.status);
    } catch (err) {
      setTasks(before); // 3. roll back, and say so
      setError('Could not update "' + task.title + '" (' + err.message + ')');
    }
  }

  return (
    <main>
      {error && <p role="alert">{error}</p>}
```

```

    <ul>{tasks.map((t) => (
      <li key={t.id}><label><input type="checkbox" checked={t.done} onChange={() => toggle(t)} /> {t.title}
    </label></li>
    ))}</ul>
  </main>
);
}

```

Example. The Try it box loads the tasks and adds one more that "someone else deleted a minute ago" (id 99). Tick any real task: the tick appears at once, and the PATCH succeeds a second later. Tick "Deleted elsewhere": the tick appears, the server answers 404, and the code rolls back with a message.

Common mistake. Rolling back to a snapshot taken before **several** overlapping requests. Tick task 1 and task 2 quickly; if the first fails, `setTasks(before)` also undoes task 2, which succeeded. For overlapping actions, roll back only the item that failed (`setTasks((prev) => prev.map(...))` for that id).

Server validation errors on the right field

When the API rejects a form with 400 { error, errors: [{ field, message }] }, turn the list into an object keyed by field ({ title: 'title must not be empty' }) and show each message next to its input, marked with `aria-invalid` and linked with `aria-describedby`.

1. On 400, turn errors: [{ field, message }] into { title: '...' } (see [One error shape](#)).
2. Show each message next to its input; errors with no field, or no matching input, go in one message at the top.
3. Mark the input `aria-invalid="true"` and point its `aria-describedby` at the message, so a screen reader reads it on focus. Colour alone is not enough.
4. Clear the errors when the user submits again.

```

import { useState } from 'react';

function App() {
  const [form, setForm] = useState({ title: '', userId: '42' });
  const [errors, setErrors] = useState({});
  const [saved, setSaved] = useState(null);
  const change = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  async function submit(e) {
    e.preventDefault();
    setErrors({});
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: form.title, userId: Number(form.userId) }),
    });
    const data = await res.json();
    if (res.status === 400) {
      const byField = {};
      for (const { field, message } of data.errors) byField[field || 'form'] = message;
      return setErrors(byField); // { title: '...', userId: '...' }
    }
    if (!res.ok) return setErrors({ form: data.error });
    setSaved(data);
  }

  const props = (name) => ({ id: name, name, onChange: change, value: form[name],
    'aria-invalid': errors[name] ? true : undefined, 'aria-describedby': errors[name] ? name + '-error' : undefined
  });

  const message = (name) => errors[name] && <span id={name + '-error'} className="error"> {errors[name]}</span>;

  return (
    <form onSubmit={submit} noValidate>
      <p><label htmlFor="title">Title</label> <input {...props('title')} />{message('title')}</p>
      <p><label htmlFor="userId">Owner</label> <select {...props('userId')}>

```

```

    <option value="1">Ana (1)</option><option value="2">Leo (2)</option><option value="42">User 42</option>
  </select>{message('userId')}</p>
  {errors.form && <p role="alert">{errors.form}</p>}
  <button>Create</button>
  {saved && <p>Created task {saved.id}: {saved.title}</p>}
</form>
);
}

```

Example. In the Try it box, submit with an empty title and the owner "User 42": the server answers 400 with two errors, and each appears under its own field (the form has `noValidate`, so the browser lets the request through). Fix both and submit: 201.

Common mistake. Showing the whole error object as one alert ("Validation failed") or `JSON.stringify(data.errors)` above the form. The user then has to work out which field is meant. Map each field to its input; only errors without a field belong at the top.

One API client

Environment variables in the front end (and why they are public)

A front-end build tool such as Vite puts variables whose names start with `VITE_` into `import.meta.env` (for example `import.meta.env.VITE_API_URL`) and writes their values into the JavaScript bundle: anyone who opens the site can read them.



A `VITE_` value is written into public JavaScript.

- **Only `VITE_` names** reach `import.meta.env`: the prefix says "this one may be public". `DATABASE_URL` or `JWT_SECRET` in the same file are not exposed, and must never be renamed to "make them work".
- **Secrets stay on the server** (see [.env files and secrets](#)); the browser gets a short-lived token after logging in.
- **One file per mode:** `.env.development` and `.env.production`, read once in the API client with a fallback. Restart the dev server after a change. Vite also gives `MODE`, `DEV` and `PROD`.
- **Create React App** used the prefix `REACT_APP_` and `process.env`: same idea, same rule.

```

# .env.development (used by npm run dev)
VITE_API_URL=/api

# .env.production (used by npm run build)
VITE_API_URL=https://api.example.com/api

# NEVER in a front-end .env: these would be readable by every visitor
# VITE_JWT_SECRET=... VITE_DB_PASSWORD=...

// src/api/client.js: read it once, with a fallback
const BASE_URL = import.meta.env.VITE_API_URL ?? '/api';
if (import.meta.env.DEV) console.log('API at', BASE_URL);

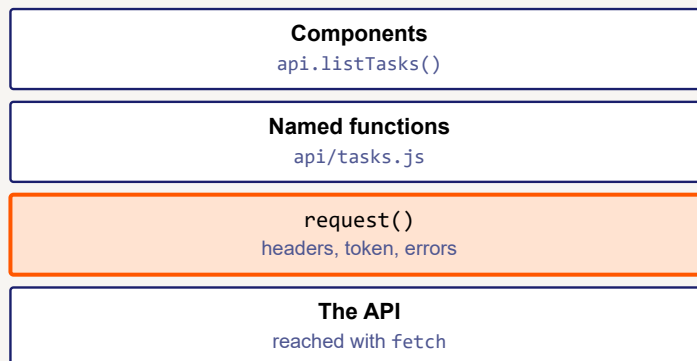
```

Example. After `npm run build`, search the generated `dist/assets/index-....js` for `api.example.com`: the URL is there as plain text. That is fine for a URL; it would be a leak for a key.

Common mistake. Putting an API key or the JWT secret in `VITE_SECRET_KEY` and believing `.env` keeps it private. `.env` is private on your disk, but the build copies every `VITE_` value into public JavaScript. If the browser needs to prove something, it must get a short-lived token from your server, never a long-lived secret.

One API client module

An **API client** is one module that every request goes through: it knows the base URL, sets the JSON headers, adds the token, checks `res.ok` and turns every failure into one error shape, so components call `api.listTasks()`, never `fetch`.



Components say what; the client says how.

One error shape An `ApiError` with status (0 for "no connection"), a human message (the server's `{ error }` when there is one) and the field errors of a 400. Components write one catch (see [Server validation errors](#)).

Two layers A generic `request(path, { method, body })` that does the HTTP, and small named functions on top (`listTasks`, `createTask`, `deleteTask`) that describe the API in words.

```

// src/api/client.js
const BASE_URL = import.meta.env.VITE_API_URL ?? '/api';

export class ApiError extends Error {
  constructor(status, message, errors = []) {
    super(message);
    this.status = status; // 0 = no response at all
    this.errors = errors; // [{ field, message }] from a 400
  }
}

export async function request(path, { method = 'GET', body, token, signal } = {}) {
  const headers = { Accept: 'application/json' };
  if (body !== undefined) headers['Content-Type'] = 'application/json';
  if (token) headers.Authorization = 'Bearer ' + token;
  let res;
  try {
    res = await fetch(BASE_URL + path, { method, headers, signal, body: body === undefined ? undefined :
JSON.stringify(body) });
  } catch (err) {
    if (err.name === 'AbortError') throw err; // cancelled on purpose
    throw new ApiError(0, 'Cannot reach the server. Check your connection.');
```

```

  }
  const data = res.status === 204 ? null : await res.json().catch(() => null);
  if (!res.ok) throw new ApiError(res.status, (data && data.error) || 'HTTP ' + res.status, (data && data.errors)
|| []);
  return data;
}

// src/api/tasks.js: the API in words
export const listTasks = (query = '', opts) => request('/tasks' + query, opts);
export const createTask = (task, opts) => request('/tasks', { method: 'POST', body: task, ...opts });
export const deleteTask = (id, opts) => request('/tasks/' + id, { method: 'DELETE', ...opts });
```

```
import { useEffect, useState } from 'react';
```

```
// ---- api.js: the only code that knows about HTTP ----
```

```

class ApiError extends Error {
  constructor(status, body) {
    super(body.error || 'HTTP ' + status);
    this.status = status;
    this.errors = body.errors || [];
  }
}

async function request(path, { method = 'GET', body } = {}) {
  const res = await fetch('/api' + path, {
    method,
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = res.status === 204 ? null : await res.json().catch(() => ({}));
  if (!res.ok) throw new ApiError(res.status, data || {});
  return data;
}

const api = {
  listTasks: () => request('/tasks'),
  createTask: (title) => request('/tasks', { method: 'POST', body: { title } }),
};

// ---- a component: only WHAT it wants ----
function App() {
  const [tasks, setTasks] = useState([]);
  const [message, setMessage] = useState('');
  useEffect(() => { api.listTasks().then(setTasks); }, []);

  async function add(title) {
    try {
      const task = await api.createTask(title);
      setTasks([...tasks, task]);
      setMessage('Created task ' + task.id);
    } catch (err) {
      setMessage(err.status + ': ' + err.message + ' ' + err.errors.map((e) => e.message).join(', '));
    }
  }

  return (
    <main>
      <button onClick={() => add('Learn fetch')}>Add "Learn fetch"</button>
      <button onClick={() => add('')}>Add an empty task</button>
      <p>{message}</p>
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </main>
  );
}

```

Example. The Try it box puts a small client and the component in one file (a real project splits them). "Add an empty task" gets a 400; the component shows the status, the message and the field error without knowing anything about HTTP.

Common mistake. Building the client but still calling `fetch` "just this once" in a page. That one call misses the token, the 401 handling and the error shape, and it is exactly the request that fails in production. Make it a rule: components import from `api/`, never call `fetch`.

fetch vs Axios: what a library adds

`fetch` is the built-in web standard; **Axios** is a popular library around HTTP requests that adds JSON by default, **rejecting on non-2xx**, **interceptors**, a **base URL** and timeouts, and both send the same HTTP.

The same needs, two tools

Need	fetch	Axios
Install	Built in	<code>npm install axios</code>
Send JSON	<code>body: JSON.stringify(x) + Content-Type header</code>	<code>axios.post(url, x)</code>
Read JSON	<code>await res.json()</code>	<code>res.data</code> (already parsed)

Need	fetch	Axios
404 or 500	Resolves; check res.ok	Rejects; err.response.status
No connection	Rejects (TypeError)	Rejects (err.response is undefined)
Base URL, default headers	Your own wrapper	axios.create({ baseUrl, headers })
Token on every request, 401 handling	Your own wrapper	Request and response interceptors
Cancel	AbortController + signal	The same signal option

```
// The same client with Axios (npm install axios). Not runnable here: the boxes have no npm.
import axios from 'axios';

export const http = axios.create({
  baseUrl: import.meta.env.VITE_API_URL ?? '/api', // every path is relative to this
});

// Request interceptor: runs before every request.
http.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = 'Bearer ' + token;
  return config;
});

// Response interceptor: runs after every response; non-2xx arrive in the error branch.
http.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) window.dispatchEvent(new Event('auth:logout'));
    return Promise.reject(error);
  },
);

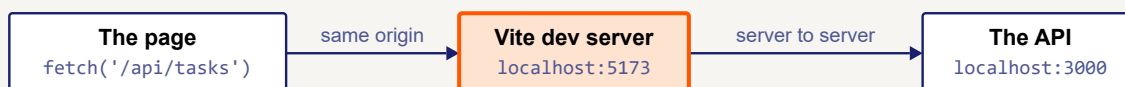
// Usage: JSON both ways, and a 404 throws.
const { data: tasks } = await http.get('/tasks', { params: { done: false } });
const { data: created } = await http.post('/tasks', { title: 'Learn Axios' });
```

Example. `await axios.get('/api/tasks/99')` rejects with `err.response.status === 404` and `err.response.data = { error: 'Task 99 not found' }`. The `fetch` version resolves; you write `if (!res.ok) throw ... yourself`. Same request on the wire, different promise behaviour.

Common mistake. Mixing the two habits: checking `res.ok` on an Axios response (it has no `ok`; it already threw), or reading `res.data` from a `fetch` response (undefined: you must `await res.json()`). Know which one a file uses, and keep it to one per project.

The dev proxy: /api → localhost:3000 without CORS

In development, the front end (Vite on `localhost:5173`) and the API (`localhost:3000`) are **different origins**; a **dev proxy** makes the dev server forward `/api/...` requests to the API, so the browser only talks to one origin and **CORS never comes into play**.



The browser only talks to its own origin; Vite forwards the rest.

Two ways to reach the API in development

	Direct: <code>fetch('http://localhost:3000/api/tasks')</code>	Proxy: <code>fetch('/api/tasks')</code>
Origin the browser talks to	<code>localhost:3000</code> (cross-origin)	<code>localhost:5173</code> (same origin)
CORS needed?	Yes, with a preflight for JSON + a token	No
Address in the code	Hard-coded or from an environment variable	Relative; the proxy decides
Network tab shows	Requests to port 3000, plus OPTIONS preflights	Requests to port 5173

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      // /api/tasks → http://localhost:3000/api/tasks
      '/api': 'http://localhost:3000',

      // If the API's routes have no /api prefix (/tasks), strip it:
      // '/api': { target: 'http://localhost:3000', changeOrigin: true,
      //           rewrite: (path) => path.replace(/^\/api/, '') },
    },
  },
});
```

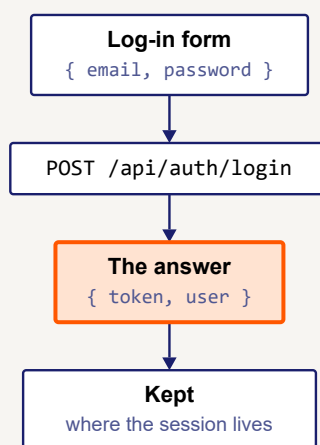
Example. Without a proxy, `fetch('http://localhost:3000/api/tasks')` from the page on port 5173 fails with "blocked by CORS policy" until the API adds `cors()`. With the proxy above and `fetch('/api/tasks')`, the same API answers and the console stays clean, with no change on the server.

Common mistake. Restarting the API, adding `Access-Control-Allow-Origin: *` everywhere, or installing a "CORS unblock" browser extension to fix a development CORS error. The proxy is the clean fix in development; in production, configure CORS for the real origin. Also remember the proxy only applies to **relative** URLs: `fetch('http://localhost:3000/...')` bypasses it.

Logging in and tokens

Logging in from the front end

Logging in is a POST of the credentials (`{ email, password }`) **to the API**; a successful answer carries a **token** (and usually the user), which the front end keeps and sends with later requests.



One POST in, a token out.

- **The form:** disable the button while the request runs; show "Invalid email or password" on 401 and the field errors on 400. The values can be read once, on submit (see [Uncontrolled inputs](#)).
- **Where to keep the token** is a trade-off between memory, `localStorage` and an `httpOnly` cookie: see [Where to keep the token](#) and, for the server side, [Sending and expiring tokens](#).
- **Never** log the password, put it in a URL (`GET /login?password=...` ends up in server logs and history) or store it: keep only the token.

```
import { useState } from 'react';
```

```
function App() {
  const [email, setEmail] = useState('ana@example.com');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [session, setSession] = useState(null); // { token, user }
  const [sending, setSending] = useState(false);

  async function login(e) {
    e.preventDefault();
    setError('');
    setSending(true);
    try {
      const res = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      setSession(data); // a real app also keeps data.token (context, storage)
    } catch (err) {
      setError(err.message);
    } finally {
      setSending(false);
    }
  }

  if (session) return <p>Welcome, {session.user.name} ({session.user.role}). Token: <code>{session.token}</code></p>;
  return (
    <form onSubmit={login}>
      <label>Email <input value={email} onChange={(e) => setEmail(e.target.value)} /></label>
      <label>Password <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} />
    </label>
    <button disabled={sending}>{sending ? 'Logging in...' : 'Log in'}</button>
    {error && <p role="alert">{error}</p>}
  </form>
  );
}
```

Example. The Try it box talks to the mock API's POST /api/auth/login. Log in as ana@example.com with password password123 (an admin) or as leo@example.com (a student). Try a wrong password (401) and an empty one (400 with field errors).

Common mistake. Telling the user "No account with that email" on one failure and "Wrong password" on another. That lets anyone find out which emails are registered. Show one message for both, as the API does: "Invalid email or password".

Sending the token: Authorization: Bearer

After logging in, the front end proves who it is on each request by sending the token in the Authorization header: Authorization: Bearer <token> (the word **Bearer**, one space, the token).

- **Add it in one place**, the API client, reading the current token from wherever the app keeps it: no component can forget it, and logging out takes effect for every request at once.
- **Cross-origin**, the header makes the request non-simple, so the browser sends a preflight first (see Preflight requests); with the dev proxy there is no cross-origin request.

```
import { useState } from 'react';

function App() {
  const [token, setToken] = useState('admin-token');
  const [result, setResult] = useState('');
```

```

async function createUser() {
  const headers = { 'Content-Type': 'application/json' };
  if (token) headers.Authorization = 'Bearer ' + token; // the word Bearer, a space, the token
  const res = await fetch('/api/users', {
    method: 'POST',
    headers,
    body: JSON.stringify({ name: 'Iris', email: 'iris' + Date.now() + '@example.com', password: 'longpassword'
  })),
  });
  const data = await res.json();
  setResult(res.status + ': ' + (res.ok ? 'created user ' + data.id : data.error));
}

return (
  <main>
    <label>Token{' '}
      <select value={token} onChange={(e) => setToken(e.target.value)}>
        <option value="admin-token">admin-token (Ana, admin)</option>
        <option value="student-token">student-token (Leo, student)</option>
        <option value="">no token</option>
        <option value="expired-token">expired-token</option>
      </select>
    </label>
    <button onClick={createUser}>POST /api/users</button>
    <p>{result}</p>
  </main>
);
}

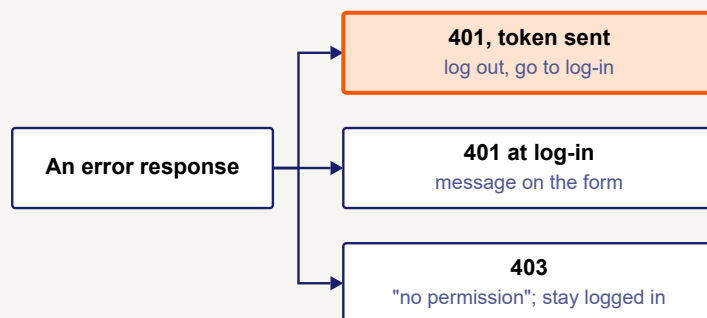
```

Example. The Try it box creates a user with POST /api/users, which only admins may do. Send it with Ana's admin token (**201**), with Leo's student token (**403**: known, not allowed), with no token and with an expired one (**401**: unknown). See [Authentication vs authorisation](#) for what each means.

Common mistake. Writing Authorization: token or Authorization: Bearer: token, or building it as 'Bearer' + token without the space. The server splits on the space and finds no token, so every request gets 401 although the user "is logged in". Log the exact header once in the Network tab.

Handling 401: log out and go to the log-in page

A **401** on a request that **carried a token** means the token is no longer accepted (expired, revoked, the secret changed): handle it in **one place**, forget the token, show the logged-out state and go to the log-in page with a short explanation.



Only a refused token logs the user out.

- **A callback, not routing:** the client stays framework-neutral with `createClient({ getToken, onUnauthorized })`. The app's `onUnauthorized` clears the session wherever the app keeps it and goes to the log-in page, remembering where the user was (see [Sharing the token with the API client](#) and [Back to where you were after log-in](#)).
- **Only when a token was sent:** a 401 from the log-in request itself means a wrong password, shown on the form.
- **403 is different:** the user is known but not allowed (see [Authentication vs authorisation](#)), so logging out would not help; show "You do not have permission".

```
import { useState } from 'react';

// The client knows HOW to spot an expired token; the app decides what logging out means.
function createClient({ getToken, onUnauthorized }) {
  return async function request(path, options = {}) {
    const token = getToken();
    const headers = { 'Content-Type': 'application/json' };
    if (token) headers.Authorization = 'Bearer ' + token;
    const res = await fetch('/api' + path, { ...options, headers });
    if (res.status === 401 && token) { // we sent a token and it was refused
      onUnauthorized();
      throw new Error('Session expired');
    }
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'HTTP ' + res.status);
    return data;
  };
}

function App() {
  const [token, setToken] = useState('expired-token'); // saved days ago
  const [notice, setNotice] = useState('');
  const request = createClient({
    getToken: () => token,
    onUnauthorized: () => { setToken(null); setNotice('Your session has expired. Please log in again.')} },
  );

  async function addUser() {
    try {
      await request('/users', { method: 'POST', body: JSON.stringify({ name: 'Iris', email: 'iris@example.com',
password: 'longpassword' }) });
      setNotice('User created. ');
    } catch (err) { console.log('request failed:', err.message); }
  }

  if (!token) return <p role="status">{notice} <button onClick={() => { setToken('admin-token'); setNotice('');
}}>Log in again</button></p>;
  return <p><button onClick={addUser}>Create a user</button> {notice}</p>;
}
```

Example. The Try it box starts with a token "saved days ago" that the server no longer accepts. Press Create a user: the client sees 401, calls onUnauthorized, and the app shows the logged-out state with an explanation. Log in again (admin token) and the request works (201; a second click gives 409 because the email now exists).

Common mistake. Redirecting to /login on **every** 401, including the one from the log-in form. A user who mistypes the password is then "redirected" to the page they are on, and the message about the wrong password never appears. Check that a token was sent (or skip the handler for the log-in call).